

CSCE 633 MACHINE LEARNING · MODULE 02 · LECTURE NOTES

Linear Models

Derived from the loss, not recited from a formula.

MODULE OUTCOMES

- Derive the normal equations from squared loss.
- Explain why the closed form is rarely used.
- Implement gradient descent and reason about its step size.
- Derive logistic regression and explain why squared loss fails for it.
- Explain convexity and what it guarantees.

1 Least squares

$$\mathbf{J}(\mathbf{w}) = \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 \rightarrow \nabla \mathbf{J} = 2\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \rightarrow \mathbf{w} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Three steps from 'minimise squared error' to the answer. The loss is convex in $\mathbf{w}$ (it is a positive semidefinite quadratic), so setting the gradient to zero finds the global minimum rather than merely a stationary point. **Most people can recite the third expression and cannot derive it**, which matters because the derivation is what generalises — the same three steps with a different loss give you a different estimator, and §3 does exactly that.

DO NOT INVERT THE MATRIX

Forming $\mathbf{X}^T\mathbf{X}$ squares the condition number of $\mathbf{X}$, so a matrix that was mildly ill-conditioned becomes badly so, and the computed solution loses roughly half its significant digits. This is a real effect on real data with correlated features.

Solve the least-squares problem directly with a QR decomposition or an SVD, both of which work on $\mathbf{X}$ itself and never form $\mathbf{X}^T\mathbf{X}$. Every numerical library does this; `lstsq` is the function, not `inv`.

And explicit inversion is worse than solving in every case. `solve(A, b)` rather than `inv(A) @ b` — always, for any linear system, in any language. The inverse is a mathematical object that is almost never the right computational one.

This is CSCE 620 Module 01's lesson in a different domain: the mathematically equivalent expression is not the numerically equivalent one. The textbook form is written for legibility and proof, not for evaluation, and taking it as an implementation specification is a recurring error across every computational subject.

	Normal equations	Gradient descent
Cost	$O(nd^2 + d^3)$ — the cube is the solve.	$O(nd)$ per iteration, times the iteration count.
Many features (large d)	The d^3 term dominates and becomes infeasible past roughly 10^4 features.	Fine.
Many samples (large n)	X must fit in memory, or be processed in a streaming $X^T X$ accumulation.	The stochastic version streams and never holds the full dataset.
Exactness	Exact, in one step.	Iterative, and needs a stopping criterion that is itself a judgement.
Other loss functions	Squared error only.	Any differentiable loss.

The last row is the decisive one. The closed form is a special case that exists because squared loss happens to have a linear gradient; it generalises to nothing. **Gradient descent generalises to every model in this course and the next one**, which is why it, rather than the elegant solution, is what the field is built on.

2 Gradient descent

```
repeat: w <- w - eta * grad_J(w)
```

THE STEP SIZE DECIDES EVERYTHING

```
too small -> converges slowly, and you conclude it is stuck
too large -> oscillates, or diverges to NaN within a few steps
just right -> depends on the curvature, which changes as you move
```

FEATURE SCALING IS NOT OPTIONAL

```
one feature over [0,1] and another over [0,1e6] makes the loss
surface a long thin valley: GD oscillates across the narrow
direction and crawls along the long one. The SAME model with
standardised features converges in a fraction of the steps.
```

VARIANTS, in order of importance:

```
stochastic / mini-batch  noisier steps, far more per second
momentum                accumulate velocity; damps oscillation
Adam                    per-parameter adaptive steps; the
                        default, and CSCE 636 M03 is about
                        when it is not
```

CONVEXITY IS WHAT MAKES THIS SAFE

A convex function has a single minimum, and any local minimum is therefore the global one. There are no other stationary points to be trapped in.

So gradient descent on a convex loss cannot converge to the wrong answer. It can converge slowly, or oscillate, or stop early — all of which are tuning problems with diagnosable symptoms, not correctness problems.

Squared loss over a linear model is convex, and so is logistic loss (§3) and the SVM's hinge loss (Module 07). The whole of classical machine learning lives in this comfortable territory, which is why the theory is as clean as it is.

Neural networks are not convex — the loss surface has enormous numbers of local minima and saddle points — which is why CSCE 636 Module 03 spends an entire module on optimisation. **The surprising empirical fact is that gradient descent works anyway**, reliably, on surfaces where the theory offers no guarantee whatsoever, **and nobody fully understands why**. It is worth knowing that this is an open question rather than a settled one.

3 Logistic regression

$$P(y=1|x) = \sigma(w^T x) \quad L = -\sum [y \log p + (1-y) \log(1-p)] \quad \nabla L = X^T(p - y)$$

The sigmoid squashes the linear score into (0,1) so it can be read as a probability; the loss is the negative log-likelihood, which is cross-entropy, and it is convex in w . **Note that the gradient has exactly the same form as least squares** — the design matrix transposed, times the residual — **which is not a coincidence**: both are generalised linear models and the pattern holds for the whole family.

WHY NOT SQUARED LOSS ON THE SIGMOID?

It is not convex. Composing the squared loss with the sigmoid produces a surface with local minima, so the guarantee of §2 is lost for no benefit.

And the gradients vanish exactly when you need them most. When the model predicts 0.99 and the true label is 0, the sigmoid is deep in its flat region, so its derivative is nearly zero and the squared loss produces almost no gradient. **The model is confidently wrong and barely corrects.**

Cross-entropy does the opposite. Its gradient grows as the prediction becomes more confidently wrong — the log term diverges — so the correction is large precisely when the error is egregious.

This is the actual reason, and it is worth knowing rather than accepting cross-entropy as a convention. The same argument recurs throughout CSCE 636: **a loss is chosen for the shape of its gradient, not only for what it measures.**

LOGISTIC REGRESSION OUTPUTS PROBABILITIES, AND THAT MATTERS

Most classifiers emit a score with no particular meaning; logistic regression emits a probability that is calibrated when the model is reasonably well specified — among examples it rates at 0.7, about 70% are positive.

So the decision threshold is yours to choose, from the relative cost of each error type (Module 01 §3), rather than fixed at 0.5 by a convention that encodes equal costs. **Moving the threshold is free and is almost always worth doing**, and it is the cheapest way to respond to class imbalance (Module 05).

And probabilities compose. They can be multiplied, used inside expected-value calculations, combined across models, and compared between systems — none of which is true of an arbitrary score.

Many stronger classifiers are badly calibrated. Boosted trees are systematically overconfident and modern neural networks are dramatically so (a finding that surprised the field). **Module 05 covers measuring and fixing this**, and it is a real reason to keep a logistic regression in the comparison even when something else wins on accuracy.

4 Why linear is still the first thing to fit

- **A baseline that is frequently hard to beat**, particularly on tabular data with a few thousand rows — which describes a very large share of real problems.
- **Coefficients you can read**. Direction, magnitude, and statistical significance, which constitutes a genuine explanation of the model's behaviour rather than a post-hoc attribution heuristic applied to a black box.
- **Calibrated probabilities** (§3), which many stronger models do not provide.
- **Training fast enough to cross-validate exhaustively**, to bootstrap confidence intervals, and to retrain on every data refresh without thinking about it.
- **And a diagnostic about the problem itself. If a linear model reaches nearly the performance of a complex one, the problem is mostly linear** — which tells you that further model capacity is not where the remaining gains are, and redirects effort toward features or data. **Fit a linear model first, always**; it costs minutes and it tells you what kind of problem you have.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS229 — supervised learning notes, parts I and II (free)	The derivations of §1 and §3 , including the generalised linear model framing that explains why the two gradients match.
James et al. — An Introduction to Statistical Learning, chapters 3 and 4 (free PDF)	Linear and logistic regression with the statistical interpretation of the coefficients, which §4 relies on.
Sebastian Ruder — An Overview of Gradient Descent Optimization Algorithms (free)	The §2 variants, compared clearly, with the update rules written out.
Trefethen & Bau — Numerical Linear Algebra	Why not to form $X^T X$ — the conditioning argument of §1, done properly. Lecture 11 is the relevant one.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Derive the normal equations from the squared loss by hand.
- 2 **Implement least squares three ways** — explicit inverse, solve, and QR — and compare on an ill-conditioned design matrix.
- 3 **Measure the digit loss**: construct X with a known condition number and compare each method against the exact answer.

- 4 Implement gradient descent and sweep the step size over four orders of magnitude. Plot convergence for each.
- 5 **Demonstrate the thin-valley problem:** fit with unscaled features, then standardised, and compare the step counts.
- 6 Implement mini-batch gradient descent and compare wall-clock convergence against full-batch.
- 7 Derive the logistic regression gradient by hand and verify it numerically.
- 8 **Fit a sigmoid with squared loss and with cross-entropy** on confidently-wrong examples, and plot the gradient magnitude for each.
- 9 Check whether your logistic regression is calibrated: bin the predictions and plot observed frequency against predicted probability.
- 10 **Fit a linear model on your project dataset** and record the score as the benchmark every later model must beat.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Derive the normal equations and say why the stationary point is the minimum.
- 2 Why should you not form $X^T X$, and what should you do instead?
- 3 Compare the closed form and gradient descent on five axes, and say which row decides it.
- 4 What does feature scaling fix, and why does it matter so much?
- 5 What does convexity guarantee, and which models have it?
- 6 Why is cross-entropy used rather than squared loss on a sigmoid?
- 7 Why do calibrated probabilities matter?
- 8 Give five reasons to fit a linear model first.